

QUALIDADE DE SOFTWARE

AULA 02

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS	11

EXEMPLO

O QUE VEM POR AÍ?

Nesta sessão, vamos aprofundar nossos conhecimentos em testes de unidade, focando em como aplicá-los eficazmente com o Jest, um framework popular para testes em projetos Node.js. Descobriremos como o Jest nos permite estruturar, escrever e executar testes de unidade, garantindo que cada componente do nosso código esteja funcionando corretamente. Vamos explorar as funcionalidades do Jest que facilitam a identificação e correção de bugs, especialmente em sistemas que passaram por atualizações significativas e começaram a apresentar falhas.

HANDS ON

Nesta aula, vamos focar na elaboração de testes de unidade para um projeto que recentemente enfrentou problemas com bugs após uma atualização. Destacaremos a importância vital de incorporar testes desde o início do desenvolvimento. Exploraremos como esses testes podem efetivamente identificar e corrigir erros, assegurando que futuras atualizações sejam minuciosamente testadas antes da implementação. Esta estratégia não só eleva a qualidade do software, mas também promove um processo de desenvolvimento mais estável e confiável.

SAIBA MAIS

Testes de unidade

O desenvolvimento de software é um processo complexo que requer a utilização de boas práticas para garantir a qualidade e a confiabilidade dos sistemas. Entre essas práticas, os testes de unidade desempenham um papel fundamental. Nesta aula abordaremos a importância dos testes de unidade, evidenciando seus conceitos básicos, benefícios e melhores práticas.

Mas, antes, o que são os testes de unidade?

Os testes de unidade são uma forma de teste de software que se concentra em verificar o funcionamento adequado de uma unidade de um código-fonte. As unidades podem ser funções, métodos, classes ou até mesmo pequenos trechos de código.

O objetivo principal dos testes de unidade é assegurar que cada unidade funcione corretamente de forma isolada antes de serem integradas em módulos maiores ou no sistema como um todo.

Existem muitos benefícios em se testar pequenas unidades de nosso código, como:

- **Deteção precoce de defeitos:** ao testar cada unidade de forma isolada, os testes de unidade ajudam a identificar erros e falhas de lógica no código de forma precoce, tornando mais fácil e rápido corrigi-los.

Isso contribui para a redução de custos e o aumento da eficiência no desenvolvimento de software.

- **Facilitação da manutenção do código:** com testes de unidade bem elaborados fica mais seguro realizar alterações no código, uma vez que é possível verificar rapidamente se as mudanças introduziram algum problema ou impacto indesejado em outras partes do sistema.

Isso torna a manutenção e a evolução do software mais ágeis e menos propensas a introduzir novos defeitos.

- **Documentação viva do código:** os testes de unidade podem servir como uma forma de documentação do código, fornecendo exemplos claros de como cada

unidade deve ser utilizada e como se comportar corretamente em diferentes situações.

Isso facilita o entendimento e o uso do código por outras pessoas especialistas em desenvolvimento, mesmo em projetos complexos ou com alta rotatividade de equipe.

- **Promoção da confiança e da qualidade do software:** ao realizar testes de unidade abrangentes e bem estruturados, é possível aumentar a confiança na estabilidade e na qualidade do software.

Isso se reflete na satisfação dos usuários finais, que terão uma experiência mais consistente e livre de erros.

Acredito que você esteja se perguntando agora se existe algum guideline (boa prática) para o desenvolvimento dos seus testes, correto?

A resposta é sim. E, nesta aula, aprenderemos algumas, como:

- **Escreva testes antes do código:** siga a abordagem do Test-Driven Development (TDD) e escreva os testes antes de implementar o código. Isso ajuda a definir os requisitos e comportamentos esperados do código, além de garantir uma cobertura adequada dos casos de teste. Caso esse seja seu primeiro contato com TDD, falaremos sobre ele posteriormente.
- **Mantenha os testes pequenos e rápidos:** testes de unidade devem ser rápidos de executar e fáceis de entender. Divida os testes em unidades lógicas menores e evite testar funcionalidades complexas em um único teste.
- **Cobertura abrangente:** procure testar todos os caminhos e condições possíveis dentro da unidade sendo testada. Isso inclui testar diferentes valores de entrada, casos de borda e cenários de erro.
- **Automatize os testes:** utilize ferramentas de automação de testes, como frameworks de teste, para facilitar a execução e a manutenção dos testes de unidade. Isso permite a execução rápida e repetitiva dos testes, além de integrá-los aos processos de build e integração contínua.
- **Teste tanto o comportamento esperado quanto o comportamento não esperado:** certifique-se de testar não apenas os casos em que o código deve funcionar corretamente, mas também aqueles em que o código deve falhar ou

gerar resultados inesperados. Isso ajuda a identificar possíveis problemas e a melhorar a robustez do código.

- **Atualize os testes conforme o código evolui:** à medida em que o código é alterado ou atualizado, os testes de unidade também devem ser atualizados para refletir as mudanças. Isso garante que os testes permaneçam relevantes e eficazes ao longo do tempo.

Seguir essas diretrizes ajudará na criação de testes de unidade eficazes e que aumentam a confiabilidade do software, facilitam a identificação de erros e contribuem para um código mais limpo e de fácil manutenção. Lembre-se de que os testes de unidade devem ser realizados em conjunto com outras formas de testes, como testes de integração e testes de sistema, para garantir uma cobertura completa e abrangente do software.

Após esta introdução, vamos explorar como criar testes de unidade usando Jest, um framework de teste popular para aplicações JavaScript e Node.js. Com Jest, podemos elaborar testes de unidade de forma clara e eficaz. Vejamos um exemplo prático de como utilizar o Jest para testar funções simples em um projeto Node.js.

Para começar, instale o Jest no seu projeto Node.js usando npm ou yarn:

```
npm install --save-dev jest
```

ou

```
yarn add --dev jest
```

Agora, vamos ver um exemplo de teste de unidade para uma função simples que soma dois números:

Crie um arquivo chamado `calculator.js`:

```
function sum(a, b) {  
  return a + b;  
}  
  
module.exports = sum;
```

Agora, crie um arquivo de teste para essa função, chamado calculator.test.js:

```
const sum = require('./calculator');  
  
test('adds 1 + 2 to equal 3', () => {  
  expect(sum(1, 2)).toBe(3);  
});
```

Neste exemplo, temos uma função sum que simplesmente soma dois números. O arquivo de teste calculator.test.js usa o Jest para definir um teste, onde expect(sum(1, 2)).toBe(3); verifica se o resultado da função sum quando chamada com os argumentos 1 e 2 é igual a 3.

Para rodar os testes, adicione o seguinte script no seu package.json:

```
"scripts": {  
  "test": "jest"  
}
```

E execute o teste com:

```
npm test
```

ou

```
yarn test
```


Este exemplo ilustra a simplicidade e eficiência do Jest para criar e executar testes de unidade em projetos JavaScript. Com Jest, você tem acesso a uma variedade de recursos úteis, como a execução de testes em paralelo, snapshots, e muito mais, facilitando a manutenção de um código limpo e funcional.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os fundamentos importantes dos testes de unidade, focando no uso do Jest, um framework popular para testar aplicações JavaScript, incluindo Node.js.

Se surgirem dúvidas, estamos à disposição no Discord. Lá, você também pode trocar ideias com colegas e professores, além de participar de nossas lives. Não perca essa chance de aprofundar seu aprendizado!

EMANDA

REFERÊNCIAS

MICROSOFT. **Noções básicas de teste de unidade**. 2023. Disponível em: <<https://learn.microsoft.com/pt-br/visualstudio/test/unit-test-basics?view=vs-2022>>. Acesso em: 12 de abr. 2024.

EXEMPLO

PALAVRAS-CHAVE

Palavras-chave: Testes de unidade. Qualidade de Software. Jest.

EMSE

POSTECH